

УДК Lorem ipsum dolor sit amet

Операционная семантика выражений в языке Go на языке ABML

Харьков А.А. (Студент ММФ НГУ)

Ануреев И.С. (Институт систем информатики СО РАН)

В данной статье представлено формальное описание операционной семантики выражений языка программирования Go с использованием языка спецификаций ABML. Предложенная онтологическая модель позволяет строго задать семантику базовых конструкций языка и может служить основой для разработки инструментов верификации программ.

Ключевые слова: операционная семантика, язык Go, язык ABML, формальные методы, онтологическое моделирование.

1. Введение

Формальное описание семантики языков программирования является фундаментальной задачей, лежащей в основе разработки компиляторов, статических анализаторов и инструментов верификации программ. Традиционные подходы к спецификации семантики, основанные на естественном языке, часто страдают от неоднозначности и неполноты, что затрудняет создание формально верифицируемых инструментов.

В данной работе рассматривается фрагмент языка программирования Go и его операционная семантика, формализованная с помощью языка спецификаций ABML (Agent-Based Modeling Language). Выбор языка Go обусловлен его возрастающей популярностью в промышленной разработке, особенно в области системного программирования и создания распределённых приложений. Язык ABML, в свою очередь, предоставляет средства для описания онтологий предметных областей и формализации правил перехода между состояниями.

2. Онтология фрагмента языка Go

Построение формальной модели семантики языка программирования требует предварительного создания онтологии — системы понятий, описывающих синтаксические конструкции и семантические сущности языка. В данном разделе на языке ABML вводятся типы объектов, соответствующие элементам фрагмента языка Go: переменным, функциям, операторам, декларациям и т.д.

Классификация и порядок изложения опираются на официальную спецификацию языка Go. Большинство наименований моделируемых объектов соответствуют их названиям в спецификации для удобства восприятия и облегчения поиска дополнительной информации.

2.1. Имена

Имена объектов в языке Go являются важнейшей категорией, связывающей синтаксические конструкции с семантическими сущностями. Для понимания приводимых определений необходимо знакомство с основными конструкциями языка ABML. Ключевое слово `typedef` вводит новый тип, конструкция `(uniont ...)` определяет объединение типов, а `mot` (Model of Type) и `cot` (Constant Object Type) определяют типы изменяемых и константных объектов соответственно.

```
(typedef "identifier"      (uniont string))
(typedef "constant name"  "identifier")
(typedef "variable name"  "identifier")
(typedef "field name"     "identifier")
(typedef "function name"  "identifier")
(typedef "parameter name" "identifier")
(typedef "method name"    "identifier")
(typedef "label name"     "identifier")
(typedef "type name"      "identifier")
```

Тип `"identifier"` представляет общее понятие имени объекта в программе. Остальные типы конкретизируют это понятие для различных категорий объектов.

2.2. Типы данных

Система типов языка Go разделяется на базовые (простые) и составные типы.

```
(typedef "type" (uniont "primitive type" "composite type"))
```

2.2.1. Базовые типы

Базовые типы включают логический, числовые и строковый:

```
(typedef "primitive type" (uniont "bool type" "numeric type" "string type"))  
(typedef "bool type"      (enumt "bool"))  
(typedef "numeric type"   (uniont "int type" "float type" "complex type"))  
(typedef "string type"    (enumt "string"))
```

Целочисленные типы разделяются на знаковые и беззнаковые с указанием разрядности:

```
(typedef "int type" (uniont "signed int type" "unsigned int type"))  
(cot "signed int type" :at "bit size" (enumt 8 16 32 64))  
(cot "unsigned int type" :at "bit size" (enumt 8 16 32 64))  
(cot "float type" :at "bit size" (enumt 32 64))  
(cot "complex type" :at "bit size" (enumt 64 128))
```

Конструкция `(enumt ...)` задаёт перечислимый тип, значениями которого являются указанные константы.

2.2.2. Составные типы

Составные типы объединяют структуры данных, построенные из базовых или других составных типов:

```
(typedef "composite type" (uniont "array type" "slice type" "struct type"  
                                "pointer type" "function type" "method type"  
                                "interface type" "map type" "channel type"))
```

Массивы имеют фиксированную длину, известную на этапе компиляции, в то время как срезы представляют собой динамические представления последовательных фрагментов

массивов:

```
(cot "array type" :at "elem type" "type" :at "len" nat)
(cot "slice type" :at "elem type" "type")
```

Структуры объединяют набор полей с различными типами. Атрибут `ordered` используется на этапе статического анализа для поддержки позиционной инициализации:

```
(cot "struct type" :at "fields" (cot :amap "field name" "type")
      :at "ordered" (listt "field name"))
```

Указатели и функциональные типы:

```
(cot "pointer type" :at "type" "type")
(cot "function type"
      :at "param types" (listt "type")
      :at "variadic type" "type"
      :at "result types" (listt "type"))
```

Тип метода включает получателя (receiver) и может использоваться в интерфейсах:

```
(cot "method type"
      :at "receiver type" "type"
      :at "param types" (listt "type")
      :at "variadic type" "type"
      :at "result types" (listt "type"))
```

Интерфейсы задают множество методов, которые должны быть реализованы. Словари (карты) и каналы завершают систему составных типов:

```
(cot "interface type" :at "methods" (cot :amap "method name" "method type"))
(cot "map type" :at "key type" "type" :at "elem type" "type")
(cot "channel type" :at "elem type" "type")
```

Направление канала не моделируется, так как для корректных программ оно не влияет

на семантику.

2.3. Ячейки памяти и значения

Моделирование памяти и значений является основой для описания операционной семантики. В силу строгой типизации языка Go каждый объект программы имеет явно определённый тип.

```
(mot "cell" :at "value" "Go value" :at "type" "type")
(typedef "Go value" (uniont "untyped constant" "typed primitive"
                        "composite value"))
```

Ячейка (`cell`) является изменяемым контейнером, хранящим значение. Тип ячейки указывает на тип хранимого значения. Переменные связываются с ячейками, а не непосредственно со значениями.

Нетипизированные константы существуют только на этапе статического анализа. В операционной семантике они преобразуются в типизированные значения:

```
(typedef "untyped constant" (uniont "rune value" bool int real string complex))
(mot "typed primitive"
  :at "type" "type"
  :at "value" (uniont "nil" "untyped constant"))
```

Значение `"nil"` типизировано — каждый ссылочный тип имеет собственный типизированный `nil`:

```
(mot "nil")
```

2.3.1. Составные значения

Составные значения включают массивы, срезы, структуры, карты, каналы, функции, методы, интерфейсы, а также ячейки (моделирующие значения указателей):

```
(typedef "composite value" (uniont "array value" "slice value" "struct value"
```

```
"map value" "interface value" "method value"
"function value" "cell" "channel value"))
```

Массив хранит элементы в виде списка ячеек, что обеспечивает возможность изменения элементов:

```
(mot "array value"
  :at "type"      "array type"
  :at "elements" (listt "cell"))
```

Срез является представлением фрагмента массива. Несколько срезов могут разделять один и тот же массив:

```
(mot "slice value"
  :at "type"      "slice type"
  :at "array"      "array value"
  :at "offset"     nat
  :at "length"     nat
  :at "capacity"  nat)
```

Структура хранит поля как отображение имён на ячейки:

```
(mot "struct value"
  :at "type"      "struct type"
  :at "fields" (mot :amap "field name" "cell"))
```

Карта хранит записи в виде отображения ключей на ячейки значений:

```
(mot "map value"
  :at "type"      "map type"
  :at "entries" (mot :amap "Go value" "cell"))
```

Канал представляет собой FIFO-очередь с буфером и очередями ожидающих горутин:

```
(mot "channel value"
```

```

:at "type"          "channel type"
:at "buffer"        (listt "Go value")
:at "send queue"    (listt "cell")
:at "receive queue" (listt "cell")
:at "closed"        bool)

```

Функция и метод хранят тип, сигнатуру, тело и замыкание — отображение захваченных переменных на ячейки:

```

(mot "function value"
  :at "type"          "function type"
  :at "signature"     "function signature"
  :at "body"          "block"
  :at "closure"       (mot :amap "variable name" "cell"))

```

Интерфейс хранит статический тип и значение. Динамический тип содержится внутри `“typed value”`:

```

(mot "interface value"
  :at "type"          "interface type"
  :at "value"         "typed value")

```

2.4. Литералы

Литералы представляют собой значения, которые могут быть непосредственно записаны в исходном коде программы:

```

(typedef "literal" (uniont "typed constant" "composite lit"
                          "function lit" "method lit"))
(typedef "composite lit" (uniont "array lit" "slice lit"
                                 "struct lit" "map lit"))

```

Литералы массивов и срезов могут содержать явное указание индексов:

```
(mot "index & elem" :at "index" nat :at "value" "expression")
(mot "array lit" :at "type" "array type" :at "elements" (listt "index & elem"))
(mot "slice lit" :at "type" "slice type" :at "elements" (listt "index & elem"))
```

Литералы структур могут использовать явные имена полей:

```
(mot "field & value" :at "field" "field name" :at "value" "expression")
(mot "struct lit" :at "type" "struct type"
  :at "fields" (listt "field & value"))
```

Литералы карт задают пары ключ-значение:

```
(mot "key & elem" :at "key" "expression" :at "value" "expression")
(mot "map lit" :at "type" "map type" :at "elements" (listt "key & elem"))
```

2.5. Выражения

Выражения в языке Go — это конструкции, которые после вычисления возвращают значение. Все выражения содержат атрибут `“type”`, значение которого вычисляется на этапе статического анализа.

```
(typedef "expression" (uniont "literal" "variable ref" "(1)" "conversion"
  "method expr" "selector expr" "index expr"
  "slice expr" "type assertion" "<−1"
  "function call" "method call" "unary expression"
  "binary expression"))
```

Ссылка на переменную представляет использование уже объявленной переменной:

```
(mot "variable ref" :at "name" "variable name" :at "type" "type")
```

Выражение в скобках влияет только на порядок вычисления:

```
(mot "(1)" :at 1 "expression" :at "type" "type")
```


Приведение типов позволяет явно преобразовывать значения:

```
(mot "conversion" :at "type" "type" :at "value" "expression") ; float64(2)
```

Метод-выражение возвращает функцию с получателем в качестве первого параметра:

```
(mot "method expr" :at "receiver type" "type" :at "name" "method name"  
      :at "type" "type")
```

Выбор поля или метода по экземпляру:

```
(mot "selector expr" :at "receiver" "expression" :at "name" "identifier"  
      :at "type" "type")
```

Индексирование применимо к массивам, срезам, картам и строкам:

```
(mot "index expr" :at "indexable" "expression" :at "index" "expression"  
      :at "type" "type")
```

Выражение среза создаёт новый срез из существующего массива, строки или среза. Параметры `“low”`, `“high”` и `“max”` задают границы. Последний параметр может отсутствовать:

```
(mot "slice expr"  
      :at "sequence" "expression"  
      :at "low"      "expression"  
      :at "high"     "expression"  
      :at "max"     "expression"  
      :at "type"    "slice type")
```

Проверка типа применяется к интерфейсным значениям:

```
(mot "type assertion" :at "interface" "expression" :at "type" "type")
```

Операция чтения из канала:

```
(mot "<-1" :at 1 "expression" :at "type" "type")
```

Вызов функции и метода:

```
(mot "function call" :at "function" "expression" :at "arguments" (listt
  "expression") :at "type" "type")
(mot "method call" :at "receiver" "expression" :at "method" "method name"
  :at "arguments" (listt "expression") :at "type" "type")
```

Унарные выражения используют цифровую нотацию для обозначения операнда:

```
(typedef "unary expression" (uniont "+1" "-1" "^1" "!1" "*1" "&1"))
(mot "+1" :at 1 "expression" :at "type" "type") ; +x == x
(mot "-1" :at 1 "expression" :at "type" "type") ; -x == -1 * x
(mot "^1" :at 1 "expression" :at "type" "type") ; bitwise complement (^)
(mot "!1" :at 1 "expression" :at "type" "type") ; logical not
(mot "*1" :at 1 "expression" :at "type" "type") ; pointer dereference
(mot "&1" :at 1 "expression" :at "type" "type") ; address of
```

Бинарные выражения также используют цифровую нотацию: **“1”** — левый операнд, **“2”** — правый:

```
(typedef "binary expression" (uniont "1||2" "1&&2" "1*2" "1/2" "1%2"
  "1<<2" "1>>2" "1&2" "1&^2" "1+2"
  "1-2" "1|2" "1^2" "1==2" "1!=2"
  "1<2" "1<=2" "1>2" "1>=2"))
```

2.6. Блоки

Блоки определяют области видимости переменных и меток. В языке Go блок представляет собой последовательность операторов, заключённую в фигурные скобки.

```
(mot "block"
  :at "statements" (listt "statement")
```

```

:at "label positions" (cot :amap "label name" nat)
:at "all variables"   (listt "variable name")
:at "decl variables"  (listt "variable name")
:at "variable cells"  (cot :amap "variable name" "cell"))

```

Атрибут `“statements”` содержит список операторов блока. Семантические атрибуты служат для управления областями видимости. Атрибут `“variable cells”` сохраняет для каждой переменной её ячейку памяти до входа в блок, что позволяет корректно обрабатывать затенение имён переменных. Атрибут `“label positions”` сопоставляет меткам их позиции в блоке, обеспечивая корректную обработку операторов перехода.

2.7. Декларации

Декларации вводят новые объекты в программу.

```

(typedef "declaration" (uniont "type decl" "const decl" "var decl"
                                "function decl" "method decl"))

```

Декларация типа связывает имя с определением типа:

```

(mot "type decl" :at "name" "type name" :at "type" "type")

```

Декларация переменных и констант:

```

(mot "const decl"
  :at "names" (listt "constant name")
  :at "types" (listt "type")
  :at "values" (listt "expression"))
(mot "var decl"
  :at "names" (listt "variable name")
  :at "types" (listt "type")
  :at "values" (listt "expression"))

```

Декларация функций и методов:

```
(mot "function decl" :at "name" "function name" :at "value" "function lit")
(mot "method decl" :at "name" "method name" :at "value" "method lit")
```

2.8. Операторы

Операторы в языке Go представляют собой конструкции, которые изменяют состояние программы, но не возвращают значений.

```
(typedef "statement" (uniont "declaration" "label" "empty stmt" "l++" "l—"
    "assignment stmt" "if stmt" "switch stmt"
    "for stmt" "return stmt" "break stmt"
    "continue stmt" "goto stmt" "defer stmt"
    "go stmt" "l<-2" "fallthrough" "select stmt"))
```

Помеченный оператор позволяет присвоить метку любому оператору программы. Пустой оператор не выполняет никаких действий:

```
(mot "label" :at "name" "label name" :at "statement" "statement")
(cot "empty stmt")
```

Операторы инкремента и декремента являются операторами, а не выражениями, что отличает Go от многих других языков:

```
(mot "l++ stmt" :at 1 "expression")
(mot "l— stmt" :at 1 "expression")
```

Операторы присваивания включают простое присваивание и составные присваивания с арифметическими и битовыми операциями:

```
(typedef "assignment stmt" (uniont "l=2" "l+=2" "l-=2" "l|=2" "l^=2"
    "l*=2" "l/=2" "l%=2" "l<=<=2"
    "l>>=2" "l&=2" "l&^=2"))
```

Условный оператор может содержать необязательную инициализирующую часть:

```
(mot "if stmt"
  :at "init"      "statement"
  :at "condition" "expression"
  :at "then"      "block"
  :at "else"      (uniont "if stmt" "block"))
```

Оператор выбора существует в двух формах: переключатель по выражению и переключатель по типу:

```
(typedef "switch stmt" (uniont "expr switch stmt" "type switch stmt"))
(cot "default")
(cot "fallthrough")
```

Оператор цикла может быть представлен в двух формах: классический цикл с условием и цикл с оператором `range`:

```
(typedef "for stmt" (uniont "for condition" "for range indexable"
  "for range map" "for range channel"))
```

Операторы передачи управления включают `return`, `break`, `continue` и `goto`:

```
(mot "return stmt"  :at 1 (listt "expression"))
(mot "break stmt"   :at 1 "label name")
(mot "continue stmt" :at 1 "label name")
(mot "goto stmt"     :at 1 "label name")
```

Оператор `defer` откладывает выполнение функции, а `go` запускает горутину:

```
(mot "defer stmt" :at 1 "expression")
(mot "go stmt"    :at 1 "expression")
```

Оператор отправки в канал и оператор `select` для работы с несколькими каналами:

```
(mot "l<-r" :at 1 "expression" :at 2 "expression")
```

```
(mot "select stmt" :at "cases" (listt "common clause"))
```

3. Операционная семантика выражений

Операционная семантика определяет, как вычисляются выражения языка Go в процессе выполнения программы. В данном разделе на языке ABML формализуются правила вычисления для различных типов выражений: литералов (массивов, срезов, структур, карт), а также функций и методов.

Вычисление выражений происходит в контексте окружения — агента, который хранит информацию о переменных, константах, функциях и типах, определённых в программе. Каждый шаг вычисления представлен атрибутивным замыканием (`aclosure`), которое задаёт атрибут `“opsem:rvalue”` для соответствующего типа выражения.

3.1. Окружение и агенты

Окружение (`“env”`) представляет собой совокупность агентов, каждый из которых моделирует состояние выполнения программы:

```
(mot "env" :at "agents" (listt "agent"))
(mot "agent"
  :at "constant value" (mot :amap "constant name" "Go value")
  :at "variable cell" (mot :amap "variable name" "cell")
  :at "function value" (mot :amap "function name" "function value")
  :at "method value" (mot :amap "type name"
                        (mot :amap "method name" "method value"))
  :at "type" (mot :amap "type name" "type")
  :at "value" "Go value")
```

Атрибуты агента включают:

- `“constant value”` — отображение имён констант на их значения;
- `“variable cell”` — отображение имён переменных на ячейки памяти;

- `“function value”` — отображение имён функций на их значения;
- `“method value”` — отображение типов и имён методов на значения методов;
- `“type”` — отображение имён типов на определения типов;
- `“value”` — последнее вычисленное агентом значение.

Базовое правило вычисления значения: если выражение уже является значением, оно возвращается без изменений:

```
(aclosure ac :attribute "opsem::rvalue" :type "Go value" :instance i :do i)
```

3.2. Литералы массивов

Вычисление литерала массива происходит в несколько этапов. Сначала вычисляется значение по умолчанию для элементов массива на основе типа элементов, затем создаётся массив из ячеек памяти, заполненных значениями по умолчанию, после чего вычисляются и подставляются явно заданные элементы.

```
(aclosure ac :attribute "opsem::rvalue" :type "array lit"
  :instance i :stage nil :do
  (update-push-aclosure ac :stage "fill defaults")
  (clear-update-eval-aclosure ac :attribute "default value by type"
    :instance (aget i "type" "elem type"))))
```

На этапе заполнения значениями по умолчанию создаётся массив требуемой длины:

```
(aclosure ac :attribute "opsem::rvalue" :type "array lit"
  :instance i :stage "fill defaults"
  :ap i "elements" es :ap i "type" at :ap at "elem type" et
  :ap (mo "pointer type" :av "type" et) ct :value v :do
  (match :v (null es) T :do (mo "array value" :av "type" at
    :av "elements" nil)
  :exit (update-push-aclosure ac :stage "evaluating" :av "left" es
    :av "array" (let ((c nil)) (dotimes (k (aget at "len") c)
      (setf c (cons (mo "cell" :av "value" v :av "type" ct) c))))))
```

```
(clear-update-eval-aclosure ac :instance (aget (car es) "value"))))
```

На этапе вычисления явных элементов происходит замена значений в соответствующих ячейках:

```
(aclosure ac :attribute "opsem::rvalue" :type "array lit"
  :instance i :stage "evaluating"
  :ap ac "left" left :ap ac "array" arr :value v :ap (car left) "index" k :do
    (let ((c arr)) (dotimes (j k (setf (car c) "value" v)) (setf c (cdr c))))
    (match :v (null (cdr left)) T
      :do (mo "array value" :av "type" (aget i "type") :av "elements" arr)
      :exit (update-push-aclosure ac :av "left" (cdr left) :av "array" arr)
      (clear-update-eval-aclosure ac
        :instance (aget (car (cdr left)) "value"))))
```

3.3. Литералы срезов

Вычисление литерала среза аналогично вычислению массива, но дополнительно определяется максимальный индекс для определения размера массива. После заполнения всех элементов создаётся значение среза, ссылающееся на созданный массив.

```
(aclosure ac :attribute "opsem::rvalue" :type "slice lit"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "build value")
    (update-push-aclosure ac :stage "fill defaults")
    (clear-update-eval-aclosure ac :attribute "default value by type"
      :instance (aget i "type" "elem type")))
```

На этапе создания значения среза формируется итоговый объект:

```
(aclosure ac :attribute "opsem::rvalue" :type "slice lit"
  :instance i :stage "build value"
  :value arr :ap i "type" stp :ap stp "elem type" etp :p (length arr) n :do
```



```

(mo "slice value"
  :av "type" (aget i "type")
  :av "array" (mo "array value"
    :av "type" (mo "array type" :av "elem type" etp :av "len" n)
    :av "elements" arr)
  :av "offset" 0
  :av "length" n
  :av "capacity" n))

```

3.4. Литералы структур

Вычисление литерала структуры включает заполнение полей значениями по умолчанию с последующей заменой на явно заданные значения. Для каждого поля создаётся ячейка памяти соответствующего типа.

```

(aclosure ac :attribute "opsem::rvalue" :type "struct lit"
  :instance i :stage nil
  :p (aget i "type" "fields") tfs :p (attributes tfs) tns :do
    (update-push-aclosure ac :stage "eval field values")
    (match :v (null ns) T :do nil
      :exit (update-push-aclosure ac :stage "filling defaults"
        :av "type field names" tns
        :av "struct value" (mo :amap "field name"
          "cell"))
      (clear-update-eval-aclosure ac :attribute "default value by type"
        :instance (aget tfs (car tns))))))

```

Финальное значение структуры создаётся на последнем этапе:

```

(aclosure ac :attribute "opsem::rvalue" :type "struct lit"
  :instance i :stage "build value" :value sv :do
    (mo "struct value" :av "type" (aget i "type") :av "fields" sv))

```

3.5. Литералы карт

Вычисление литерала карты происходит путём последовательного вычисления пар ключ-значение. Каждая пара обрабатывается отдельным замыканием, после чего создаётся итоговая карта.

```
(aclosure ac :attribute "opsem::rvalue" :type "map lit" :instance i :stage nil
  :ap i "elements" es :do
    (update-push-aclosure ac :stage "build value")
    (match :v (null es) T :do nil
      :exit (update-push-aclosure ac :stage "evaluating"
        :av "entries" (mo :amap "Go value" "cell")
        :ap "elements" (cdr es))
      (clear-update-eval-aclosure ac :instance (car es)
        :av "elem type" (aget i "type" "elem type")))))
```

Для вычисления отдельной записи используется замыкание `keyed elem`, которое вычисляет ключ и значение, после чего создаёт ячейку для значения:

```
(aclosure ac :attribute "opsem::rvalue" :type "keyed elem"
  :instance i :stage nil :value k :do
    (update-push-aclosure ac :stage "eval value" :av "key" k)
    (clear-update-eval-aclosure ac :instance (aget i "value")))
```

3.6. Литералы функций и методов

Функциональные литералы создают значения функций и методов, сохраняя текущее состояние замыкания — ячейки памяти всех переменных, видимых в момент создания.

```
(aclosure ac :attribute "opsem::rvalue" :type "function lit" :instance i
  :agent a :p (aget i "body" "all variables") ns :p nil c :do
    (dolist (item ns) (setf c (aset c (aget a "variable cell" item))))
    (mo "function value" :av "type" (aget i "type")
      :av "signature" (aget i "signature"))
```

```

:av "body"      (aget i "body")
:av "closure"   c))

```

Аналогично для методов:

```

(aclosure ac :attribute "opsem::rvalue" :type "method lit" :instance i
  :agent a :p (aget i "body" "all variables") ns :p nil c :do
  (dolist (item ns) (setf c (aset c (aget a "variable cell" item)))))
(mo "method value" :av "type"      (aget i "type")
  :av "signature" (aget i "signature")
  :av "body"      (aget i "body")
  :av "closure"   c))

```

3.7. Декларации

Декларации вводят новые объекты в окружение программы. Для каждого типа декларации определено атрибутивное замыкание с атрибутом `“opsem”`.

3.7.1. Декларация типа

Декларация типа добавляет определение типа в окружение агента:

```

(aclosure ac :attribute "opsem" :type "type decl" :instance i :agent a :do
  (aset a "type" (aget i "name") (aget i "type")))
)

```

3.7.2. Декларация переменных

Декларация переменных может происходить двумя способами: с явным указанием значений или без них (в этом случае используются значения по умолчанию для соответствующих типов).

```

(aclosure ac :attribute "opsem" :type "var decl" :instance i :stage nil
  :ap i "names" ns :ap i "values" vs :ap i "types" ts :do

```

```

(match :v (null vs) T
:do (update-push-aclosure ac :stage "declarating defaults"
      :av "names" ns :av "types" (cdr ts))
    (clear-update-eval-aclosure ac :attribute "default value by type"
      :instance (car ts))
:exit (update-push-aclosure ac :stage "evaluating values"
      :av "names" ns :av "types" ts :av "values" (cdr vs))
    (clear-update-eval-aclosure ac :instance (car vs)))
)

```

На этапе заполнения значениями по умолчанию создаётся ячейка памяти для каждой переменной:

```

(aclosure ac :attribute "opsem" :type "var decl"
:instance i :stage "declarating defaults" :value v :agent a
:ap ac "names" ns :ap ac "types" ts :do
(aset a "variable cell" (car ns) (mo "cell" :av "value" v
      :av "type" (mo "pointer type" :av "type" (car ts))))
(match :v (null ts) T
:exit (update-push-aclosure ac :av "names" (cdr ns) :av "types" (cdr ts))
    (clear-update-eval-aclosure ac :attribute "default value by type"
      :instance (car ts)))
)

```

Аналогично для явно заданных значений:

```

(aclosure ac :attribute "opsem" :type "var decl"
:instance i :stage "evaluating values" :value v :agent a
:ap ac "names" ns :ap ac "values" vs :ap ac "types" ts :do
(aset a "variable cell" (car ns) (mo "cell" :av "value" v
      :av "type" (car ts)))
(match :v (null vs) T
:exit (update-push-aclosure ac :av "names" (cdr ns) :av "types" (cdr ts))
)

```

```

                                :av "values" (cdr vs))
      (clear-update-eval-aclosure ac :instance (car vs)))
    )

```

3.7.3. Декларация функций и методов

Декларация функции вычисляет значение функционального литерала и сохраняет его в окружении:

```

(aclosure ac :attribute "opsem" :type "function decl"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "decl")
    (clear-update-eval-aclosure ac :instance (aget i "value")))
)
(aclosure ac :attribute "opsem" :type "function decl"
  :instance i :stage "decl" :value v :agent a :do
    (aset a "function value" (aget i "name") v)
)

```

Декларация метода сохраняется в отображении `"method value"`, где ключом является тип получателя, а затем имя метода:

```

(aclosure ac :attribute "opsem" :type "method decl"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "decl")
    (clear-update-eval-aclosure ac :instance (aget i "value")))
)
(aclosure ac :attribute "opsem" :type "method decl"
  :instance i :stage "decl"
  :value v :agent a :do
    (aset a "method value" (aget i "value" "type" "receiver type")
      (aget i "name") v)
)

```

3.8. Блоки

Блоки определяют области видимости переменных. При входе в блок сохраняются старые ячейки всех переменных, объявленных в этом блоке (для последующего восстановления при выходе). Затем последовательно выполняются операторы блока.

```
(aclosure ac :attribute "opsem" :type "block" :instance i :stage nil
  :ap i "statements" sts :ap i "decl variables" avs :p nil dv :agent a :do
  (aset i "variable cells" (dolist
    (item avs (reverse dv))
    (setf dv (cons (aget a "variable cell" item) dv))))
  (update-push-aclosure ac :stage "variable back")
  (clear-update-eval-aclosure ac :stage "evaluating statement" :av
    "statements" sts)
)
```

Вычисление очередного оператора:

```
(aclosure ac :attribute "opsem" :type "block"
  :instance i :stage "evaluating statement"
  :ap ac "statements" sts :v (null sts) nil :do
  (update-push-aclosure ac :av "statements" (cdr sts) :av "block" i)
  (clear-update-eval-aclosure ac :instance (car sts))
)
```

Восстановление старых ячеек при выходе из блока:

```
(aclosure ac :attribute "opsem" :type "block"
  :instance i :stage "variable back" :do
  (dolist (item (aget i "decl variables")) (aset a "variable cell" item
    (aget item "variable cells")))
)
```

3.9. Выражения

3.9.1. Ссылка на переменную

Для ссылки на переменную определены два замыкания: для получения левого значения (lvalue) — ячейки памяти, и для получения правого значения (rvalue) — содержимого ячейки.

```
(aclosure ac :attribute "opsem::lvalue" :type "variable ref"
  :instance i :agent a :do
    (aget a "variable cell" i)
)
(aclosure ac :attribute "opsem::rvalue" :type "variable ref"
  :instance i :agent a :do
    (aget (aget a "variable cell" i) "value")
)
```

3.9.2. Выражение в скобках

Выражение в скобках вычисляет своё содержимое:

```
(aclosure ac :attribute "opsem::rvalue" :type "(1)" :instance i :do
  (clear-update-eval-aclosure ac :instance (aget i 1))
)
```

3.9.3. Приведение типов

Приведение типов (конверсия) вычисляет значение и затем преобразует его к указанному типу (преобразование не детализировано):

```
(aclosure ac :attribute "opsem::rvalue" :type "conversion"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "conversion")
    (clear-update-eval-aclosure ac :instance (aget i "value")))
)
(aclosure ac :attribute "opsem::rvalue" :type "conversion"
```

```

:instance i :stage "conversion"

:value v :do ; TODO

)

```

3.9.4. Метод-выражение

Метод-выражение преобразует метод в функцию, где получатель становится первым параметром:

```

(aclosure ac :attribute "opsem::rvalue" :type "method expr" :instance i
  :agent a :p (aget a "method value" (aget i "receiver type")
    (aget i "name"))) mv
:ap mv "type" mt :ap mv "signature" ms :do
(mo "function value"
  :av "type" (mo "function type"
    :av "param types" (cons (aget mt "receiver type")
      (aget mt "param types")))
    :av "variadic type" (aget mt "variadic type")
    :av "result type" (aget mt "result types"))
:av "signature" (mo "function signature"
  :av "parameters" (cons (aget ms "receiver")
    (aget ms "parameters")))
    :av "variadic param" (aget ms "variadic param")
    :av "result" (aget ms "result"))
:av "body" (aget mv "body")
:av "closure" (aget mv "closure"))
)

```

3.9.5. Выбор поля или метода

Выбор поля или метода (selector expression) обрабатывает два случая: когда слева от точки стоит имя типа (интерфейса) — тогда возвращается метод; когда слева стоит структура

или указатель на структуру — тогда возвращается поле.

Для `“opsem::rvalue”`:

```
(aclosure ac :attribute "opsem::rvalue" :type "selector expr"
  :instance i :stage nil :ap i "receiver" r :do
    (update-push-aclosure ac :stage "return value")
    (match :v (and (is-instance r "identifier") (aget a "type" r)) T
      :do (aget a "method value" r (aget i "name"))
      :exit (clear-update-eval-aclosure ac :instance (aget i "receiver"))))
)
```

Для `“opsem::lvalue”` (только для структур):

```
(aclosure ac :attribute "opsem::lvalue" :type "selector expr"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "return value")
    (clear-update-eval-aclosure ac :instance (aget i "receiver")
      :attribute "opsem::rvalue")
)
(aclosure ac :attribute "opsem::lvalue" :type "selector expr"
  :instance i :stage "return value"
  :value r :ap i "name" n :ap r "type" tr :do
    (nmatch :v tr "struct type" :exit (aget r "fields" n)
      :v tr "pointer type" :exit (aget r "target" "value" "fields" n))
)
```

3.9.6. Индексирование

Индексирование применимо к массивам, срезам, картам и строкам. Для `“opsem::rvalue”` возвращается значение элемента; для `“opsem::lvalue”` — ячейка памяти (кроме строк, которые неизменяемы).

```
(aclosure ac :attribute "opsem::rvalue" :type "index expr"
```

```

:instance i :stage nil :do
  (update-push-aclosure ac :stage "eval index")
  (clear-update-eval-aclosure ac :instance (aget i "indexable"))
)

```

Возврат значения по индексу:

```

(aclosure ac :attribute "opsem::rvalue" :type "index expr"
  :instance i :stage "return value"
  :value index :ap ac "indexable" indl :ap indl "type" it :do
    (nmatch :v it "array type" :exit (aget (nth index (aget indl "elements"))
      "value"))
      :v it "slice type" :exit (aget (nth (+ (aget indl "offset") index)
        (aget indl "array" "elements")) "value"))
      :v it "map type" :exit (aget (aget indl "entries" index)
        "value"))
      :v it "string type" :exit (char (aget indl "value" index)))
)

```

3.9.7. Выражение среза

Выражение среза создаёт новый срез из существующего массива, среза или строки. Последовательно вычисляются параметры `“low”`, `“high”` и `“max”` (последний может отсутствовать).

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage nil :do
    (update-push-aclosure ac :stage "eval low")
    (clear-update-eval-aclosure ac :instance (aget i "sequence"))
)

```

В зависимости от типа последовательности создаётся соответствующий срез. Для массива:

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"

```

```

:instance i :stage "array type"

:ap ac "sequence" s :ap ac "low" low :ap ac "high" :ap ac "cap" cap
:ap s "type" st :p (length (aget s "elements")) len :do
(if (not high) (aset high len))
(if (not cap) (aset cap (− len low)))
(mo "slice value"
  :av "type" (mo "slice type" :av "elem type" (aget st "elem type")))
  :av "array" s
  :av "offset" low
  :av "length" (− high low)
  :av "capacity" cap)
)

```

Для среза (сохраняется ссылка на исходный массив):

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage "slice type"
  :ap ac "sequence" s :ap ac "low" low :ap ac "high" high :ap ac "cap" cap
  :p (length (aget s "array" "type" "len")) len :do
  (if (not high) (aset high len))
  (if (not cap) (aset cap (− len low)))
  (mo "slice value"
    :av "type" (aget s "type")
    :av "array" (aget s "array")
    :av "offset" (+ (aget s "offset") low)
    :av "length" (aget s "length")
    :av "capacity" (aget s "capacity")))
)

```

Для строки возвращается подстрока (неизменяемая):

```

(aclosure ac :attribute "opsem::rvalue" :type "slice expr"
  :instance i :stage "string type"
  :p (aget ac "sequence" "value") s :ap ac "low" low :ap ac "high" high :do

```

```
(if (not high) (aset high (length s)))
(subseq s low high))
```

3.10. Итоговая структура операционной семантики

Разработанная операционная семантика охватывает следующие категории конструкций:

- Литералы массивов, срезов, структур и карт;
- Функциональные литералы и замыкания;
- Декларации типов, переменных, функций и методов;
- Блоки и области видимости;
- Выражения: ссылки на переменные, скобки, приведения типов, метод-выражения, выбор полей и методов, индексирование, срезы.

Каждое правило представлено в виде атрибутного замыкания `aclosure`, что позволяет использовать механизмы сопоставления с образцом для управления последовательностью вычислений и обработки различных вариантов синтаксических конструкций.

4. Заключение

В работе представлена формальная операционная семантика выражений языка Go, реализованная на языке спецификаций ABML. Разработанная модель включает:

- Онтологическое описание синтаксических конструкций языка Go, включая систему типов, выражения, операторы и декларации;
- Формализацию понятий памяти и значений с использованием типизированных ячеек (`“cell”`);
- Атрибутные замыкания (`aclosure`), задающие правила вычисления для каждого типа выражений;
- Механизмы работы с замыканиями функций и методов, сохраняющие контекст выполнения;
- Правила вычисления для литералов массивов, срезов, структур и карт;
- Правила декларации типов, переменных, функций и методов;

- Правила обработки областей видимости (блоков) с поддержкой затенения переменных;
- Семантику выражений: ссылки на переменные, приведения типов, метод-выражения, выбор полей и методов, индексирование, срезы.

Предложенный подход демонстрирует применимость языка ABML для формального описания семантики языков программирования. Разработанная модель может служить основой для:

- Построения инструментов статического анализа и верификации программ на Go;
- Автоматической генерации тестов на основе формальной семантики;
- Создания интерпретаторов и компиляторов с формально верифицированной семантикой;
- Обучения и документирования языковых конструкций Go.

Дальнейшие исследования могут быть направлены на расширение модели для охвата большего числа конструкций языка Go, включая конкурентные механизмы (горутины и каналы), обобщённые типы (дженерики), а также разработку методов автоматической верификации свойств программ, основанных на предложенной семантике.